COMP2522

Lesson 12: Design Patterns

Five Important Design Patterns

- **Singleton** (Jason)

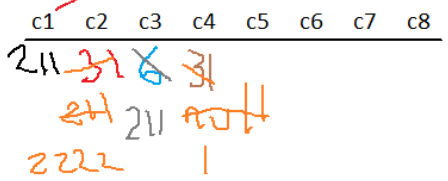
Each set will cover a design pattern

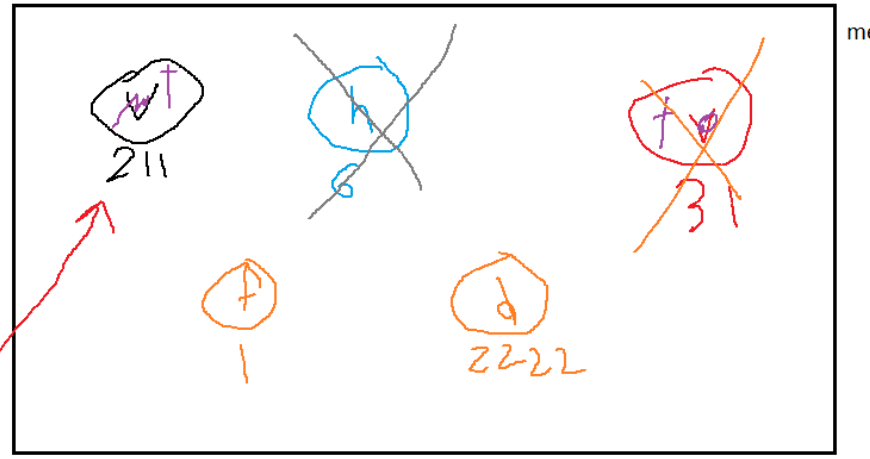
- Each set member uploads the same powerpoint, code, lab, and quiz

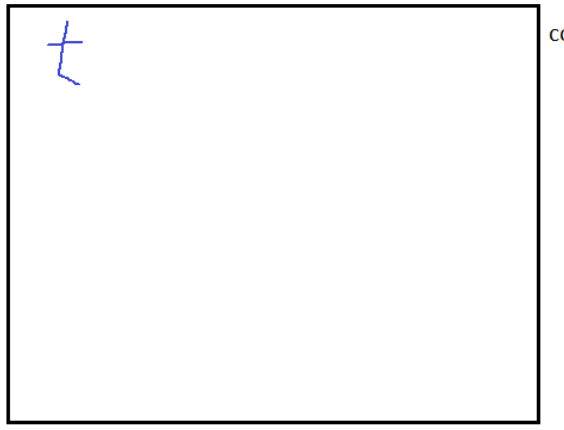
References vs. Objects

```
Car c1 = new Car("volvo");
Car c2 = new Car("volvo");
Car c3 = new Car("honda");
Car c4 = c2;
c1.setMake("toyota");
c2.setMake("toyota");
c3 = c1;
sop(c4.getMake();

c2 = c3;
c4 = null;

or

c4 = new Car("ford");
c2 = new Car("dodge");
```

code







c1 is a reference == pointer == alias == address

When there are zero references to an object, that object will be marked automatically for garbage collection.

Design Pattern #1 of 6: Singleton

- Sometimes multiple objects of a class are neither needed nor wanted
- Examples: expensive remote database connections; log files
- We don't want hundreds of connections to a log file; just create one and other requests will piggyback on that single connection
- This saves resources and time
- Problem: whenever a constructor is called an object reference is returned and a new object gets created in memory
- Solution: make the constructor private (therefore uncallable outside the class)
- New Problem: now ZERO objects can be constructed outside of that class
- Second Solution: make a static method which acts like a "substitute constructor" but which creates a new object (by calling the private constructor) if and only if there is no existing object; otherwise it simply returns a reference to the alreadyexisting object.

Why Singleton?

1. Configuration Settings: manage application configuration settings that are shared across different components of an application.
A Singleton ensures that changes to the settings are consistently reflected throughout the application.

2. Logger Class: Implementing a logging mechanism where all parts of an application write to the same log file.
A Singleton Logger ensures that all log messages are centralized and managed through a single instance.

3. Database Connection Pool: Managing a pool of database connections.
The Singleton pattern ensures that the connection pool is created only once and is globally accessible, which helps in reusing the connections efficiently.

4. Caching: Implementing a global cache that stores frequently-accessed data.
A Singleton cache allows different parts of an application to access and update the cached data consistently.

5. Thread Pool: Managing a thread pool where threads are reused for executing tasks.
A Singleton thread pool ensures that the pool is initialized only once and that all tasks are managed through a single pool of threads.

6. Resource Manager: Handling access to shared resources, such as file systems or hardware devices.
A Singleton Resource Manager ensures that resource allocation and access are coordinated and that resource conflicts are minimized.

7. Service Locator: Implementing a Service Locator that provides a central registry of services
A Singleton Service Locator ensures that all services are registered and accessed through a single instance, simplifying service management and lookup.

Singleton in Action

```
public class DbConnection
{
	private static DbConnection d;

	static
	{
		d = null;  // singletons have 0 or 1 objects only
	}

	private DbConnection(final String host, final String user, final String pass)
	{
		// do normal constructor stuff here
	}

	public static DbConnection getInstance(final String host, final String user, final String pass)
	{
		if(d == null)
		{
				d = new DbConnection(host, user, pass);   // there was no object, so make one
		}
		return d; // return the existing object
	}
}
```

Singleton Lab and Quiz

- Lab:
- Use a Singleton Design Pattern to create a class called LogFile
- No matter how many requests are made to create LogFile objects, only a single instance (maximum) will ever exist
- Quiz:
- Why would we use a Singleton design pattern? Give a real-life example. Log file, expensive database connections.
- What are the important ingredients to implement a singleton? Private constructor, nonprivate static substitute constructor which returns a reference to a static variable. This substitute constructor will call the private constructor if no object yet exists.

Real-life Example

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    private Connection connection;

    private final String URL = "jdbc:mysql://localhost:3306/your_database_name";
    private final String USER = "your_username";
    private final String PASSWORD = "your_password";

     private DatabaseConnection() {
        try {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Database connection established.");
        } catch (final SQLException e) {
            System.err.println("Database connection failed: " + e.getMessage());
        }
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            synchronized (DatabaseConnection.class) { // Thread-safe initialization
                if (instance == null) {
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }
    public Connection  getConnection() {
        return connection;
    }
}
```